

PARLE TILAK VIDYALAYA ASSOCIATION'S
MULUND COLLEGE OF COMMERCE (AUTONOMOUS)
SCHOOL OF COMPUTING & TECHNOLOGY

A PROJECT REPORT

ON

AI-DRIVEN SMART HEALTHCARE ANALYTICS: DIABETES RISK PREDICTION
SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of
BACHELOR OF SCIENCE (COMPUTER SCIENCE / INFORMATION TECHNOLOGY /
COMPUTER APPLICATIONS)

OR

BACHELOR OF COMPUTER APPLICATIONS

By: _____

Roll Number: _____

Under the guidance of

Project Guide

MULUND 400080, MAHARASHTRA
ACADEMIC YEAR 2025-26

COLLEGE CERTIFICATE

Certificate will be issued by the college. Insert signed and stamped certificate page here.

LIVE PROJECT CERTIFICATE (IF APPLICABLE)

This project is an academic capstone project. If the department requires a live-project certificate, insert it here.

PROFORMA FOR PROJECT APPROVAL

Attach the original copy of the approved project proposal as provided in department proforma.

ACKNOWLEDGEMENT

I sincerely thank the In-charge Principal, the School of Computing & Technology, my project guide, faculty members, and all supporting staff for their guidance and support. I also thank my family and friends for their encouragement throughout the project duration.

DECLARATION

I hereby declare that this project report is my original work. All sources and references used in this report have been acknowledged properly. This work has not been copied in part or full from any other source.

Student Signature: _____

ABSTRACT

This project presents a complete machine-learning based healthcare analytics system for diabetes risk prediction. The solution includes data preprocessing, model training and evaluation, web-based deployment, database-backed user interaction tracking, and visual analytics. The system accepts clinical input parameters such as Glucose, BMI, Blood Pressure, Age, and Diabetes Pedigree Function to estimate risk. The deployment includes authentication, prediction history, downloadable reports, and data-driven charts, making the platform suitable for educational research and capstone demonstration.

INDEX (SEMESTER VI CAPSTONE FORMAT)

Note: Final page numbers can vary after printing and formatting adjustments.

CHAPTER 1: INTRODUCTION

1.1 Background

Diabetes is a major chronic condition requiring early and continuous risk screening. This capstone builds an AI-driven analytical platform that predicts diabetes risk from clinical inputs and provides a full-stack user experience including prediction history and downloadable reports.

1.2 Objectives

- Build an end-to-end ML workflow from data preprocessing to deployment.
- Compare baseline models and select the best model based on ROC-AUC.
- Develop secure user interaction modules (login, registration, prediction history).
- Implement a database-backed system for reports and prediction records.
- Provide visual analytics and downloadable prediction outputs.

1.3 Purpose, Scope, and Applicability

Purpose: To provide a practical healthcare AI screening tool for diabetes risk prediction.

Scope: Covers data ingestion, model training, web API development, dashboard UI, and persistent user interactions.

Applicability: Useful for academic demonstrations, basic clinical triage support, and learning healthcare analytics workflows.

1.4 Organization of Report

This report is organized into survey of technologies, requirements analysis, system design, implementation/testing, results, conclusion, references, and annexure sections.

CHAPTER 2: SURVEY OF TECHNOLOGIES (10 SELECTED PAPERS)

The selected references collectively support model development, evaluation, deployment, and visualization strategy for the capstone system.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Manual diabetes risk assessment is time-consuming and not always accessible. The project solves this through a real-time predictive system with historical tracking and reporting.

3.2 Requirements Specification

3.3 Planning and Scheduling

3.4 Software and Hardware Requirements

3.5 Preliminary Product Description

The product is a web-based healthcare analytics dashboard with login/register, prediction engine, model insights, patient dataset exploration, report analysis, and downloadable prediction files.

CHAPTER 4: SYSTEM DESIGN

4.1 UML Diagrams

- Use Case Diagram: Patient login, predict risk, download reports.
- Activity Diagram: Input -> Validation -> Model Inference -> Save -> Download.
- Sequence Diagram: Browser -> Flask API -> Model -> SQLite -> Response.
- Class/Component View: Data utilities, training module, API layer, template layer.

4.2 Module Design

4.3 Database Design

The project uses SQLite for persistent user interaction tracking. Three core tables are designed:

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Overview of the Implementation

Implementation follows modular software engineering principles. The backend is implemented in Flask, the ML pipeline in scikit-learn, and the frontend in vanilla web technologies.

5.2 Development Tools and Technology Stack

5.3 System Architecture

Client browser interacts with Flask endpoints. Flask validates input and invokes the trained model artifact. Predictions and reports are persisted in SQLite for future retrieval and export.

5.4 Module-wise Implementation

5.4.1 User Interface Design: Dark themed responsive dashboard with role-neutral navigation.

5.4.2 Functional Modules: Predict risk, analyze report, patient data exploration, charts.

5.4.3 Database Integration: Users, predictions, and reports stored in SQLite.

5.4.4 Input/Output Handling: JSON and multipart input with numeric and schema validation.

5.5 Challenges Faced and Solutions Implemented

5.6 Testing Strategy and Methodology

Unit and integration tests are implemented using pytest/unittest style scripts from the tests folder. Core test focus: preprocessing behavior, training artifact correctness, endpoint behavior, and authentication guards.

CHAPTER 6: RESULTS

6.1 Test Reports

Project Visual Outputs:

Figure: confusion_matrix.png

Figure: correlation_matrix.png

Figure: roc_curve.png

Figure: target_distribution.png

6.2 User Documentation

1. Open the app: <http://127.0.0.1:5000/>
2. Register a user account or login using existing credentials.
3. Navigate to Predict Risk and fill the clinical input fields.
4. Click Analyze Risk to generate prediction and probability.
5. Use 'Download Latest Predicted File' or 'Download History CSV'.
6. Review model insights and patient record analytics sections.

Implemented API Endpoints:

CHAPTER 7: CONCLUSION

7.1 Significance of the System

The project demonstrates a complete capstone-ready healthcare analytics workflow: data engineering, model selection, deployment, user interaction persistence, and downloadable outputs.

7.2 Limitations of the System

The current model is trained on a compact benchmark/synthetic-compatible dataset and should be treated as an academic decision-support prototype, not a clinical diagnostic instrument.

7.3 Future Scope of the Project

Future scope includes richer clinical features, model explainability dashboards, role-based access control, cloud deployment, and integration with secure healthcare information systems.

REFERENCES / BIBLIOGRAPHY

1. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
2. Cox, D. R. (1958). The Regression Analysis of Binary Sequences.
3. Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters.
4. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python.
5. Flask Documentation. <https://flask.palletsprojects.com/>
6. SQLite Documentation. <https://sqlite.org/docs.html>
7. Chart.js Documentation. <https://www.chartjs.org/docs/>
8. NIDDK / Pima Indians Diabetes Database resources.
9. Hastie, Tibshirani, Friedman. The Elements of Statistical Learning.
10. Project source files: src/app.py, src/train.py, src/Utils/*.py, tests/*.py.

ANNEXURE

1. List of Tables

Table A: Requirements Specification

Table B: Planning and Scheduling

Table C: Technology Stack

Table D: Database Design

Table E: Challenges and Solutions

Table F: Test Cases

Table G: Model and Dataset Metrics

Table H: API Route Inventory

2. List of Diagrams/Figures

Figure 1: confusion_matrix.png

Figure 2: correlation_matrix.png

Figure 3: roc_curve.png

Figure 4: target_distribution.png

3. List of Screenshots

- Dashboard Home
- Prediction Form and Result Panel
- Model Insights Section
- Patient Records Table
- Prediction History and Download Actions

4. Plagiarism Report

Attach the official plagiarism report generated as per department submission process.

Field	Details
Student Name	_____
Roll Number	_____
Class	TYBSc/ TYBCA
Department	School of Computing & Technology
Project Title	AI-Driven Smart Healthcare Analytics: Diabetes Risk Prediction System

Sr. No.	Chapter Scheme	Pages
1	CHAPTER 1: INTRODUCTION	1-4
2	CHAPTER 2: SURVEY OF TECHNOLOGIES (10 Papers)	5-9
3	CHAPTER 3: REQUIREMENTS AND ANALYSIS	10-14
4	CHAPTER 4: SYSTEM DESIGN	15-20
5	CHAPTER 5: IMPLEMENTATION AND TESTING	21-30
6	CHAPTER 6: RESULTS	31-36
7	CHAPTER 7: CONCLUSION	37-39
8	REFERENCES / BIBLIOGRAPHY	40-41
9	ANNEXURE	42-45

Sr	Paper / Source	Key Contribution	Project Relevance
1	Cox (1958) - Logistic Regression	Established logistic modeling for binary outcomes	Used as baseline model
2	Breiman (2001) - Random Forests	Ensemble learning with strong generalization	Used as alternate candidate
3	Pima Indians Dataset (NIDDK)	Clinical benchmark dataset for diabetes prediction	Core dataset reference
4	Fawcett (2006) - ROC Analysis	Threshold-independent classifier evaluation	Used for model comparison
5	Bishop (Pattern Recognition)	ML fundamentals and model validation	Guided feature handling
6	Hastie et al. (ESL)	Statistical learning framework	Supported model-selection process
7	Scikit-learn User Guide	Pipeline and cross-validation best practices	Implemented in training flow
8	Flask Documentation	Web backend route architecture	Used for API and templates
9	SQLite Documentation	Lightweight relational persistence	Enabled user-interaction database
10	Chart.js Documentation	Data visualization in web dashboard	Used for analytics charts

Type	Requirement
Functional	User registration and login
Functional	Predict diabetes risk from 8 clinical features
Functional	Save and retrieve prediction history per user
Functional	Upload/analyze report files and save output
Functional	Download individual prediction report and full CSV export
Non-Functional	Fast API response for inference requests
Non-Functional	Data persistence using SQLite database
Non-Functional	Readable dashboard with charts and tables

Phase	Major Tasks	Status
Phase 1	Dataset preparation and preprocessing utilities	Completed
Phase 2	Model training pipeline and selection	Completed
Phase 3	Flask API and dashboard integration	Completed
Phase 4	User interaction DB and report downloads	Completed
Phase 5	Testing, documentation, and black book	Completed

Category	Details
Programming Language	Python 3.13
Backend	Flask + flask-cors
ML Libraries	scikit-learn, pandas, numpy, joblib
Frontend	HTML/CSS/JavaScript, Chart.js
Database	SQLite
OS	Windows 10/11 (project tested on Windows)
Minimum Hardware	8 GB RAM, dual-core CPU, 500 MB free storage

Module	Description
Data Factory	Synthetic dataset generation fallback when real CSV is absent
Preprocess	Zero-to-missing normalization and split utilities
Train Pipeline	Model comparison, CV scoring, artifact generation
Prediction API	Input validation, risk probability, save interaction
User Management	Registration/login with hashed password storage
Reports Module	Upload analysis, report storage, retrieval
Dashboard	Stats, charts, patient table, prediction history

Table	Important Columns
users	username (PK), password_hash, first_name, last_name, email, role, created_at
predictions	id (PK), username, submitted_at, prediction, risk_probability, patient fields, feature values
reports	id (PK), submitted_by, submitted_at, patient_name, age, clinical fields, notes

Layer	Tools / Files
Backend	src/app.py (Flask routes, authentication, DB interaction)
Model Training	src/train.py (CV model selection, metrics, artifact save)
Data Utilities	src/utls/preprocess.py, src/utls/data_factory.py
UI Templates	src/templates/index.html, login.html, register.html
Testing	tests/test_train_and_app.py, tests/test_preprocess.py
Database	SQLite file: data/user_interactions.db

Challenge	Solution
Missing dataset in fresh environment	Added deterministic synthetic dataset generator fallback
Persistent user tracking requirement	Migrated CSV interaction storage to SQLite database
Test isolation with custom env paths	Database path fallback linked to PREDICTIONS_PATH in tests
Download requirement for predicted data	Added single-file and full-history export endpoints
UI consistency across pages	Unified branding and color theme, shared logo asset

Test Case	Expected Result	Status
GET /health	Returns status=ok when model is loaded	Passed
POST /predict without auth	Returns HTTP 401	Passed
POST /predict with valid payload	Returns prediction and risk_probability	Passed
GET /api/predictions/summary	Returns count/latest/previous	Passed
POST /api/analyze-report	Parses report and saves interaction	Passed
Preprocess zero handling	Converts selected zeros to NaN	Passed

Metric	Value
Dataset Rows	768
Dataset Columns	9
Outcome Count (0)	207
Outcome Count (1)	561
Selected Model	logistic_regression
Cross-Validation ROC-AUC	0.8435
Accuracy	0.8117
Precision	0.8487
Recall	0.9018
F1 Score	0.8745
ROC-AUC	0.8852

Sr	Route Definition
1	@app.get(/)
2	@app.route(/login, methods=[GET, POST])
3	@app.route(/register, methods=[GET, POST])
4	@app.get(/logout)
5	@app.get(/health)
6	@app.get(/doc)
7	@app.get(/docs)
8	@app.post(/predict)
9	@app.post(/api/analyze-report)
10	@app.get(/api/predictions)
11	@app.get(/api/predictions/summary)
12	@app.get(/api/predictions/export)
13	@app.get(/api/predictions/download)
14	@app.get(/api/model-insights)
15	@app.get(/api/patients)
16	@app.post(/api/add-report)
17	@app.get(/api/reports)